

使用 MCP Toolbox for AlloyDB 和 ADK 建構多代理應用程式

來源：<https://codelabs.developers.google.com/multi-agent-app-toolbox-adk?hl=zh-tw>

步驟數：14

在本程式碼研究室中，您將為廚房翻修專案建構多代理應用程式，以便處理翻修提案建立、處理許可和法規遵循要求，以及透過 Vertex AI Agent Development Kit 與 AlloyDB 整合的訂單管理。在這個實作中，我們會使用 AlloyDB 的 MCP Toolbox，將訂單資料擷取步驟從代理程式和工具中抽離。

目錄

1. [1. 總覽](#)
2. [2. MCP](#)
3. [3. 事前準備](#)
4. [4. 設定 ADK](#)
5. [5. 專案結構](#)
6. [6. 原始碼](#)
7. [7. 資料庫設定](#)
8. [8. 設定 MCP Toolbox for Databases](#)
9. [9. 模型設定](#)
10. [10. 設定環境變數](#)
11. [11. 執行代理程式](#)
12. [12. 結果](#)
13. [13. 清除所用資源](#)
14. [14. 恭喜](#)

1. 總覽

代理是自主程式，會與 AI 模型對話，並使用手邊的工具和情境資訊執行目標導向的作業，還能根據事實自主決策！

如果應用程式有多個代理，可視需要自主協同運作，以達成更大的目標，且每個代理都具備獨立知識，並負責特定領域，那麼應用程式就會成為**多代理系統**。

Agent Development Kit (ADK)

Agent Development Kit (ADK) 是彈性十足的模組化框架，可用於開發及部署 AI 代理。ADK 支援將多個不同的代理例項組合成多代理系統 (MAS)，建構出複雜的應用程式。

在 ADK 中，多代理系統是指應用程式內的不同代理 (通常會形成階層) 協作或協調，以達成更大的目標。以這種方式建構應用程式可帶來顯著優勢，包括提升模組化、專業化、可重複使用性、可維護性，以及使用專用工作流程代理定義結構化控制流程的能力。

多代理系統注意事項

首先，請務必充分瞭解每位代理商的專業領域和原因。「你知道為什麼需要特定子代理程式來處理某件事嗎？」請先解決這個問題。

第二，如何將這些回應與根代理程式整合，以便進行路徑設定及解讀。

第三，您可以參閱這份說明文件，瞭解多種類型的服務專員轉送。請確認哪一個適合應用程式流程。此外，您還需要哪些各種情境和狀態，才能控制多代理系統的流量控制。

建構項目

我們將使用 MCP Toolbox for AlloyDB 和 ADK，建構多代理系統來處理廚房整修事宜。

1. Renovation Proposal Agent
2. 許可證和法規遵循檢查代理
3. 訂單狀態檢查 (使用 MCP Toolbox for Databases 的工具)

翻修提案代理程式，用來生成廚房翻修提案文件。

許可證和法規遵循代理人，負責處理許可證和法規遵循相關工作。

訂單狀態檢查代理程式，可透過在 AlloyDB 中設定的訂單管理資料庫，檢查物料的訂單狀態。不過，就資料庫部分而言，我們會使用 **MCP Toolbox for AlloyDB** 實作訂單的狀態擷取邏輯。

步驟 2 · 約 0 分鐘

2. MCP

MCP 是 Model Context Protocol 的縮寫，這項開放標準由 Anthropic 開發，可讓 AI 代理以一致的方式連結外部工具、服務和資料。這項技術基本上是 AI 應用程式的通用標準，可讓應用程式與不同資料來源和工具無縫互動。

1. 它採用用戶端/伺服器模型，其中 AI 應用程式 (主機) 會執行 MCP 用戶端，與 MCP 伺服器通訊。
2. 當 AI 代理程式需要存取特定工具或資料時，會將結構化要求傳送至 MCP 用戶端，再由 MCP 用戶端轉送至適當的 MCP 伺服器。
3. 讓 AI 模型存取外部資料和工具，不必為每個整合項目編寫自訂程式碼。
4. 簡化以大型語言模型 (LLM) 為基礎建構代理程式和複雜工作流程的程序。

MCP Toolbox for Databases

Google 的 MCP Toolbox for Databases 是資料庫專用的開放原始碼 MCP 伺服器。這項服務的設計宗旨是提供企業級的生產品質。這項服務可處理連線集區和驗證等複雜作業，讓您更輕鬆、快速且安全地開發工具。

讓代理程式存取資料庫中的資料！做法：

簡化開發作業：在不到 10 行的程式碼中將工具整合至代理、在多個代理或架構之間重複使用工具，以及更輕鬆地部署新版工具。

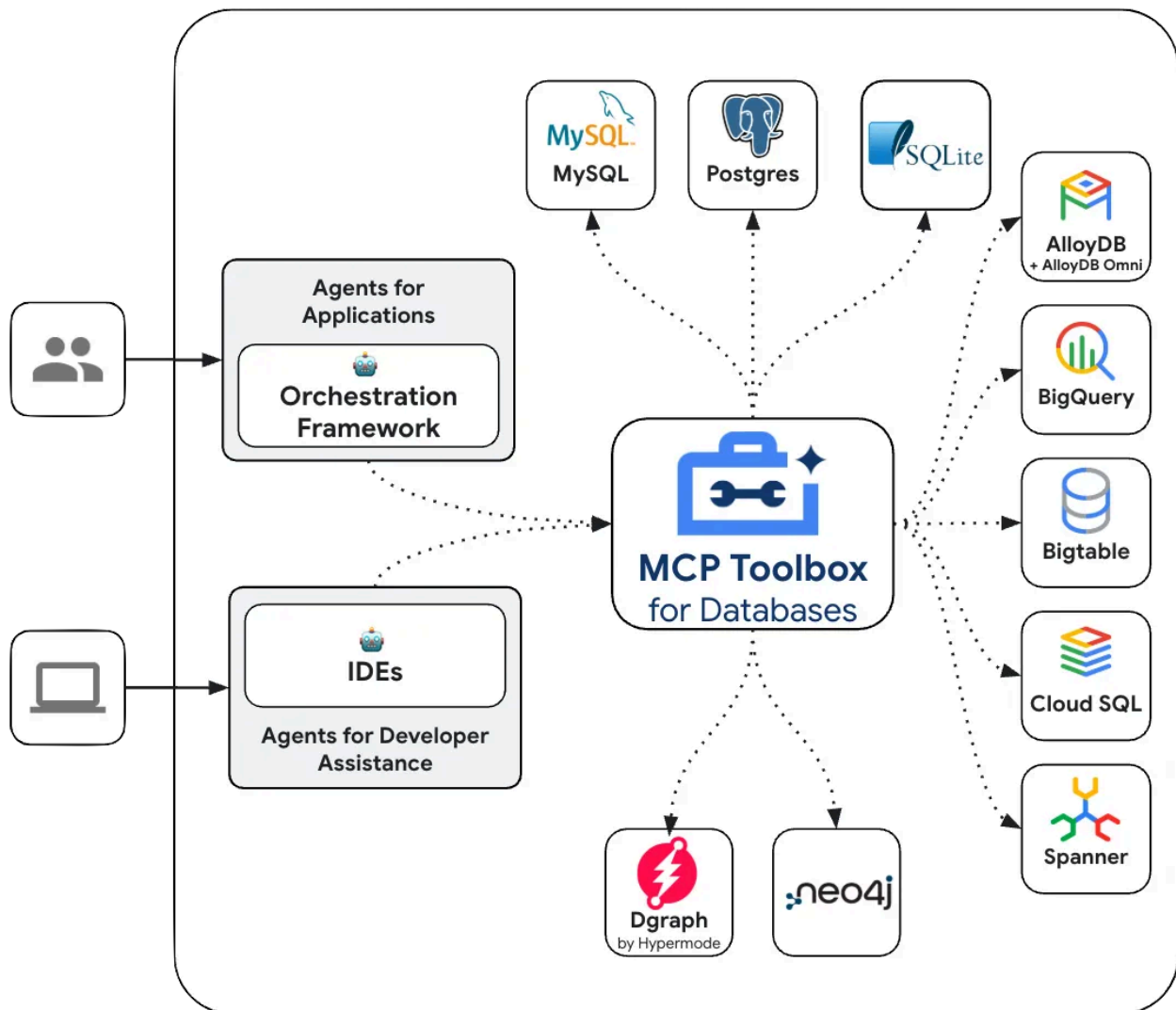
提升效能：連線集區、驗證等最佳做法。

提升安全性：整合驗證機制，確保資料存取安全無虞

端對端可觀測性：內建支援 OpenTelemetry，可提供立即可用的指標和追蹤功能。

必須指出這項功能早於 MCP 推出！

資料庫專用的 MCP Toolbox 位於具備代理功能的應用程式的自動化調度管理框架和資料庫之間，提供用於修改、發布或叫用工具的控制層。這項功能提供集中式位置來儲存及更新工具，方便您管理工具，並在代理程式和應用程式之間共用工具，以及更新工具，不必重新部署應用程式。



我們會根據需求，讓根代理程式自動調度管理這些代理程式。

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 已啟用計費功能的 Google Cloud 專案。

3. 事前準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 按一下這個[連結](#)，啟動 Cloud Shell。如要在 Cloud Shell 終端機 (用於執行雲端指令) 和編輯器 (用於建構專案) 之間切換，請點選 Cloud Shell 中的對應按鈕。
4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 執行下列指令，啟用下列 API：

```
gcloud services enable artifactregistry.googleapis.com \cloudbuild.googleapis.com  
\run.googleapis.com \aiplatform.googleapis.com \alloydb.googleapis.com
```

8. 請務必使用 Python 3.9 以上版本
 9. 如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。
-

步驟 4 · 約 0 分鐘

4. 設定 ADK

1. 建立及啟動虛擬環境 (建議)

在 Cloud Shell 終端機中建立虛擬環境：

```
python -m venv .venv
```

啟動虛擬環境：

```
source .venv/bin/activate
```

2. 安裝 ADK

```
pip install google-adk
```

步驟 5 · 約 0 分鐘

5. 專案結構

3. 在 Cloud Shell 終端機中，依序執行下列指令，建立根資料夾和專案資料夾：

```
mkdir agentic-apps
cd agentic-apps
mkdir renovation-agent
```

4. 前往 Cloud Shell 編輯器，建立檔案 (一開始為空白)，然後建立下列專案結構：

```
renovation-agent/
  __init__.py
  agent.py
  .env
```

步驟 6 · 約 0 分鐘

6. 原始碼

1. 前往 `init.py`，並更新為下列內容：

```
from . import agent
```

2. 前往 `agent.py`，並使用下列路徑中的內容更新檔案：

```
https://github.com/AbiramiSukumaran/renovation-agent-adk-mcp-toolbox/blob/main/agent.py
```

在 `agent.py` 中，我們會匯入必要的依附元件、從 `.env` 檔案擷取設定參數，並定義 `root_agent`，該代理會使用 1 項工具來叫用工具箱工具。

3. 前往 `requirements.txt`，並使用下列內容更新：

```
https://github.com/AbiramiSukumaran/renovation-agent-adk-mcp-toolbox/blob/main/requirements.txt
```

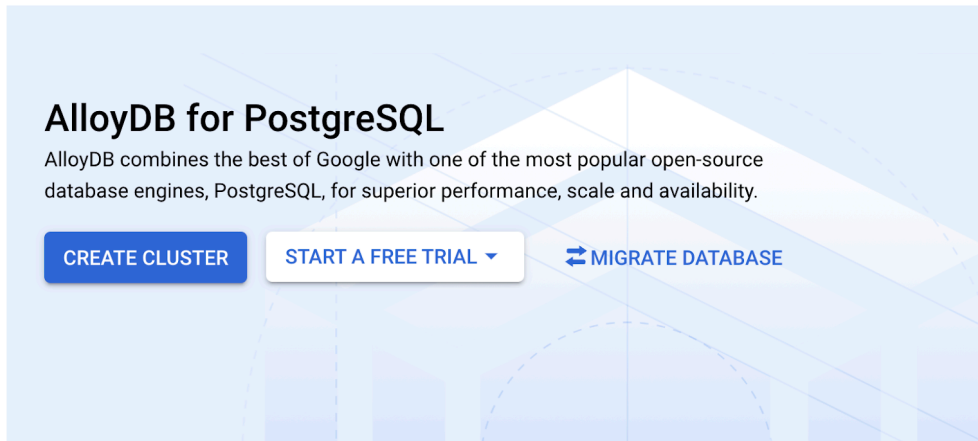
步驟 7 · 約 0 分鐘

7. 資料庫設定

在 `ordering_agent` 使用的其中一項工具 (名為「`check_status`」) 中，我們會存取 AlloyDB 訂單資料庫，以取得訂單狀態。在本節中，我們將設定 AlloyDB 資料庫叢集和執行個體。

建立叢集和執行個體

1. 在 Cloud 控制台中前往 AlloyDB 頁面。如要在 Cloud 控制台尋找大部分的頁面，只要使用控制台的搜尋列搜尋即可。
2. 選取該頁面中的「建立叢集」：



3. 您會看到如下所示的畫面。使用下列值建立叢集和執行個體 (如果您要從存放區複製應用程式程式碼，請確保值相符)：
- 叢集 ID：「`vector-cluster`」
 - password：「`alloydb`」
 - 與 PostgreSQL 16 相容 / 建議使用最新版本
 - Region：「`us-central1`」
 - 網路：「`default`」

Configure your cluster

Cluster ID *

Use lowercase letters, numbers, and hyphens. Start with a letter.

Password *



[GENERATE](#)

Set a password for the default "postgres" user. A password is required for the user to log in.

Database version *

PostgreSQL 15 compatible



Location

For better performance, keep your data close to the services that need it. Choice is permanent.

Region *

us-east4 (Northern Virginia)



Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *



[SHOW ADVANCED OPTIONS](#)

4. 選取預設網路後，你會看到如下畫面。

選取「設定連線」。

Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *

default



Private services access connection required

Your network "default" requires a private services access connection. This connection enables your services to communicate exclusively by using internal IP addresses. [Learn more](#)

[SET UP CONNECTION](#)

5. 然後選取「使用系統自動分配的 IP 範圍」並繼續。確認資訊後，選取「建立連結」。

✓ **Enable Service Networking API**

2 **Allocate an IP range**

Google will use this allocated IP range to create subnets.

☐ Select one or more existing IP ranges or create a new one

Select or create an IP range ▼

☒ Use an automatically allocated IP range

Google will automatically allocate an IP range of prefix-length /20 and use the name "default-ip-range".

CONTINUE

3 **Create a connection**

CREATE CONNECTION

CANCEL

6. 重要注意事項：請務必將執行個體 ID (您可以在設定叢集 / 執行個體時找到) 變更為

vector-instance。如果無法變更，請記得在所有後續參照中使用例項 ID。

7. 準備設定 Toolbox 時，請先在 AlloyDB 執行個體中啟用公開 IP 連線，讓新工具可以存取資料庫。

8. 前往「公開 IP 連線」部分，勾選「啟用公開 IP」核取方塊，然後輸入 Cloud Shell 機器的 IP 位址。

9. 如要取得 Cloud Shell 機器的 IP，請前往 Cloud Shell 終端機並輸入 ifconfig。從結果中找出 eth0 inet 位址，並將最後 2 位數替換為 0.0，遮罩大小為「/16」。例如「XX.XX.0.0/16」，其中 XX 為數字。

10. 將這個 IP 貼到編輯執行個體頁面的「授權外部網路」文字方塊中。

Public IP connectivity

☒ Enable public IP

Connect your applications and clients over the public Internet. May require additional configuration to ensure a secure connection. [Learn more](#) 



For the most secure connectivity, use AlloyDB connectors or AlloyDB Auth Proxy. Alternatively, make sure that your client IPs or networks are added as authorised external networks. [Learn more](#) 



Public IP requires password complexity to be enabled. We will be enabling the following flag:
`password.enforce_complexity`

Authorised external networks

Networks



Enter a range with a mask size between /16 and /32 bits.

4. 設定網路後，即可繼續建立叢集。按一下「CREATE CLUSTER」，完成叢集設定，如下所示：

Configure your primary instance

Name your instance

Give your instance an ID.

Instance ID *
shopping-cluster-primary

Choice is permanent. Use lowercase letters, numbers, and hyphens. Start with a letter.

Zonal availability

- ☐ Single zone
In case of outage, no failover. Not recommended for production.
- ☒ Multiple zones (Highly available)
Automatic failover to another zone within your selected region. Recommended for production instances. Increases cost.

Machine

AlloyDB's superior performance lets you run your existing workloads on smaller machines.

Machine Type *
8 vCPU, 64 GB

Connectivity

Private IP Connectivity

- ☒ Enable Private IP (always enabled)
Connect from applications and clients within the associated network. [Learn more](#)

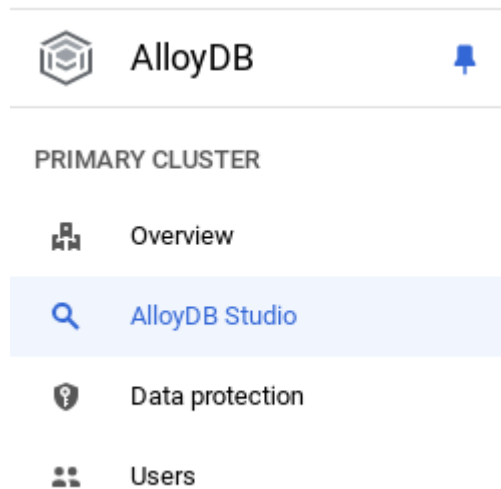
CREATE CLUSTER

CANCEL

請注意，建立叢集約需 10 分鐘。成功後，畫面上會顯示您剛建立的叢集總覽。

資料擷取

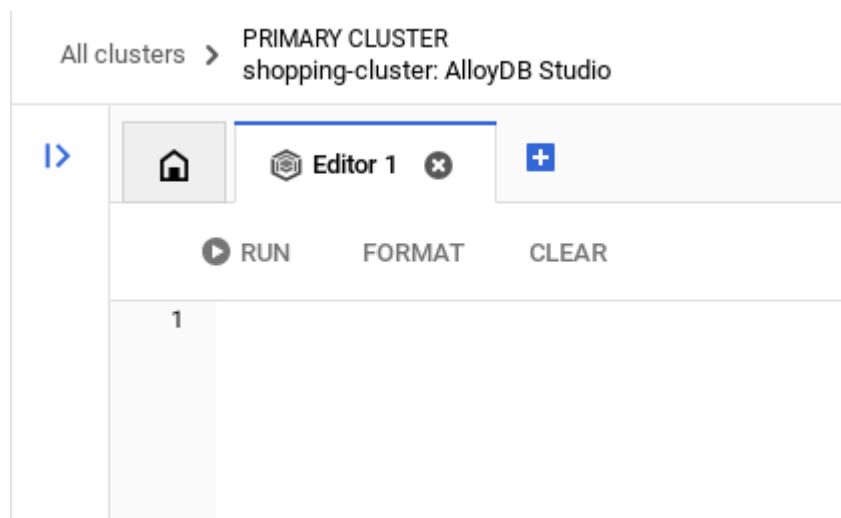
現在要新增包含商店資料的表格。前往 AlloyDB，選取主要叢集，然後選取 AlloyDB Studio：



您可能需要等待執行個體建立完成。完成後，請使用建立叢集時建立的憑證登入 AlloyDB。使用下列資料向 PostgreSQL 進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」

成功驗證 AlloyDB Studio 後，即可在編輯器中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下最後一個視窗右側的加號。



您會在編輯器視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

建立資料表

您可以在 AlloyDB Studio 中使用下列 DDL 陳述式建立資料表：

```
-- Table DDL for Procurement Material Order Status

CREATE TABLE material_order_status (
  order_id VARCHAR(50) PRIMARY KEY,
  material_name VARCHAR(100) NOT NULL,
  supplier_name VARCHAR(100) NOT NULL,
  order_date DATE NOT NULL,
  estimated_delivery_date DATE,
  actual_delivery_date DATE,
  quantity_ordered INT NOT NULL,
  quantity_received INT,
  unit_price DECIMAL(10, 2) NOT NULL,
  total_amount DECIMAL(12, 2),
  order_status VARCHAR(50) NOT NULL, -- e.g., "Ordered", "Shipped", "Delivered",
  "Cancelled"
  delivery_address VARCHAR(255),
  contact_person VARCHAR(100),
  contact_phone VARCHAR(20),
  tracking_number VARCHAR(100),
  notes TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  quality_check_passed BOOLEAN, -- Indicates if the material passed quality control
  quality_check_notes TEXT,      -- Notes from the quality control check
  priority VARCHAR(20),          -- e.g., "High", "Medium", "Low"
  project_id VARCHAR(50),        -- Link to a specific project
  receiver_name VARCHAR(100),    -- Name of the person who received the delivery
  return_reason TEXT,            -- Reason for returning material if applicable
  po_number VARCHAR(50)          -- Purchase order number
);
```

插入記錄

從上述 `database_script.sql` 指令碼複製 `insert` 查詢陳述式，並貼到編輯器。

按一下「執行」。

資料集已備妥，現在請設定 MCP Toolbox for Databases，做為 AlloyDB 中所有訂單資料庫互動的控制層！

8. 設定 MCP Toolbox for Databases

Toolbox 位於應用程式的協調架構和資料庫之間，提供用於修改、發布或叫用工具的控制層。這項功能提供集中式位置來儲存及更新工具，方便您管理工具，並在代理程式和應用程式之間共用工具，以及**更新工具 (不一定需要重新部署應用程式)**。

您會看到 MCP Toolbox for Databases 支援的資料庫之一是 AlloyDB，由於我們已在上一節中佈建該資料庫，請繼續設定 Toolbox。

1. 前往 Cloud Shell 終端機，確認已選取專案，且專案顯示在終端機的提示中。在 Cloud Shell 終端機執行下列指令，前往專案目錄：

```
cd adk-renovation-agent
```

2. 執行下列指令，在新的資料夾中下載並安裝工具箱：

```
# see releases page for other versions
export VERSION=0.7.0
curl -O https://storage.googleapis.com/mcp-toolbox-for-
databases/v$VERSION/linux/amd64/toolbox
chmod +x toolbox
```

3. 前往 Cloud Shell 編輯器 (程式碼編輯模式)，在專案根資料夾中新增名為「tools.yaml」的檔案

```
sources:
  alloydb-orders:
    kind: "alloydb-postgres"
    project: "<<YOUR_PROJECT_ID>>"
    region: "us-central1"
    cluster: "<<YOUR_ALLOYDB_CLUSTER>>"
    instance: "<<YOUR_ALLOYDB_INSTANCE>>"
    database: "<<YOUR_ALLOYDB_DATABASE>>"
    user: "<<YOUR_ALLOYDB_USER>>"
    password: "<<YOUR_ALLOYDB_PASSWORD>>"

tools:
  get-order-data:
    kind: postgres-sql
    source: alloydb-orders
    description: Get the status of an order based on the material description.
    parameters:
      - name: description
        type: string
        description: A description of the material to search for its order status.
    statement: |
      select order_status from material_order_status where lower(material_name) like
lower($1)
      LIMIT 1;
```

在查詢部分 (請參閱上方的「陳述式」參數)，我們只會在物料名稱與使用者的搜尋文字相符時，擷取訂單狀態欄位的值。

讓我們瞭解 tools.yaml。

來源代表工具可互動的不同資料來源。來源代表工具可互動的資料來源。您可以在 tools.yaml 檔案的來源區段中，將來源定義為對應。一般來說，來源設定會包含連線及與資料庫互動所需的任何資訊。

工具會定義代理可執行的動作，例如讀取及寫入來源。工具代表代理程式可執行的動作，例如執行 SQL 陳述式。您可以在 tools.yaml 檔案的 tools 區段中，將工具定義為對應。一般來說，工具需要來源才能執行動作。

如要進一步瞭解如何設定 tools.yaml，請參閱這份[說明文件](#)。

執行 MCP Toolbox for Databases 伺服器

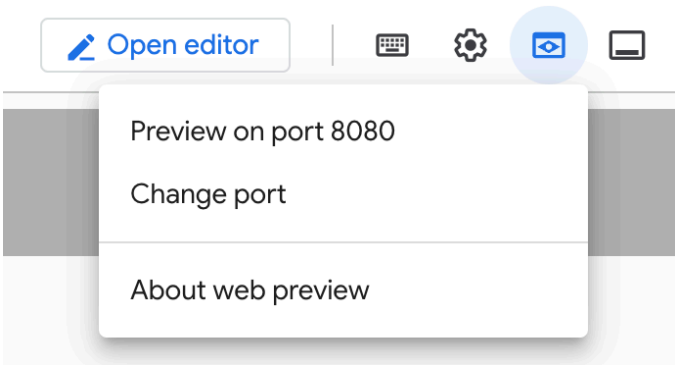
執行下列指令 (從 mcp-toolbox 資料夾) 啟動伺服器：

```
./toolbox --tools-file "tools.yaml"
```

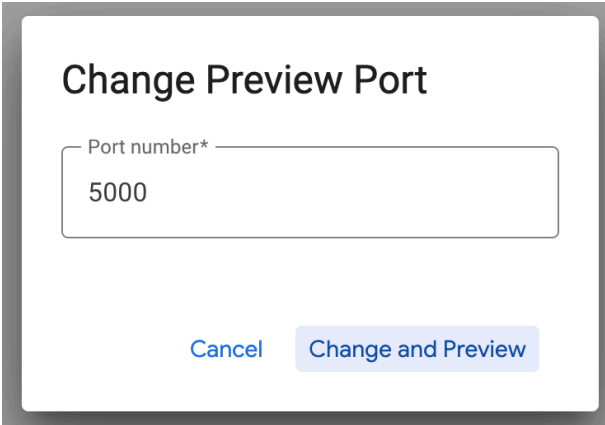
現在，如果您在雲端以網頁預覽模式開啟伺服器，應該就能看到 Toolbox 伺服器已啟動並執行，且其中包含名為 get-order-data 的新工具。

MCP Toolbox 伺服器預設會在通訊埠 5000 上執行。我們將使用 Cloud Shell 測試這項功能。

在 Cloud Shell 中點選「網頁預覽」，如下所示：



按一下「變更通訊埠」，將通訊埠設為 5000 (如下所示)，然後按一下「變更並預覽」。



這應該會產生以下輸出內容：

 Hello, World! 

MCP Toolkit for Databases 說明如何使用 Python SDK 驗證及測試工具，詳情請參閱[這篇文章](#)。我們將略過這個步驟，直接進入下一節的 Agent Development Kit (ADK)，並使用這些工具。

將 Toolbox 部署至 Cloud Run

首先，我們可以從 MCP Toolbox 伺服器開始，並將其託管於 Cloud Run。這樣一來，我們就能取得公用端點，並與任何其他應用程式和/或代理程式應用程式整合。如需在 Cloud Run 上代管這項服務的操作說明，請參閱[這篇文章](#)。我們現在來瞭解重要步驟。

1. 啟動新的 Cloud Shell 終端機，或使用現有的 Cloud Shell 終端機。前往工具箱二進位檔和 tools.yaml 所在的專案資料夾，在本例中為 adk-renovation-agent
2. 設定 PROJECT_ID 變數，指向您的 Google Cloud 專案 ID。

```
export PROJECT_ID="<<YOUR_GOOGLE_CLOUD_PROJECT_ID>>"
```

3. 啟用下列 Google Cloud 服務

```
gcloud services enable run.googleapis.com \
                        cloudbuild.googleapis.com \
                        artifactregistry.googleapis.com \
                        iam.googleapis.com \
                        secretmanager.googleapis.com
```

4. 我們將建立個別的服務帳戶，做為要在 Google Cloud Run 部署的 Toolbox 服務身分。

```
gcloud iam service-accounts create toolbox-identity
```

5. 我們也會確保這個服務帳戶具備正確角色，也就是有權存取 Secret Manager 並與 AlloyDB 通訊

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
  --role roles/secretmanager.secretAccessor

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
  --role roles/alloydb.client

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
  --role roles/serviceusage.serviceUsageConsumer
```

6. 我們將以 Secret 形式上傳 tools.yaml 檔案：

```
gcloud secrets create tools --data-file=tools.yaml
```

如果您已有密鑰，並想更新密鑰版本，請執行下列指令：

```
gcloud secrets versions add tools --data-file=tools.yaml
```

將環境變數設為要用於 Cloud Run 的容器映像檔：

```
export IMAGE=us-central1-docker.pkg.dev/database-toolbox/toolbox/toolbox:latest
```

7. 最後一個步驟是使用熟悉的部署指令將應用程式部署至 Cloud Run：

```
gcloud run deploy toolbox \
--image $IMAGE \
--service-account toolbox-identity \
--region us-central1 \
--set-secrets "/app/tools.yaml=tools:latest" \
--args="--tools-file=/app/tools.yaml", "--address=0.0.0.0", "--port=8080" \
--allow-unauthenticated
```

這會開始將 Toolbox 伺服器連同設定的 tools.yaml 部署到 Cloud Run。部署成功後，您應該會看到類似以下的訊息：

```
Deploying container to Cloud Run service [toolbox] in project [YOUR_PROJECT_ID] region [us-central1]
OK Deploying new service... Done.

OK Creating Revision...

OK Routing traffic...

OK Setting IAM Policy...

Done.

Service [toolbox] revision [toolbox-00001-zsk] has been deployed and is serving 100 percent of traffic.
Service URL: https://toolbox-<SOME_ID>.us-central1.run.app
```

您已完成設定，現在可以在代理程式應用程式中使用新部署的工具了！

讓我們將 Toolbox 工具連結至 Agent！

我們已為代理應用程式建立來源。讓我們更新該檔案，加入剛才在 Cloud Run 中部署的新 MCP Toolbox for Databases 工具。

1. 觀察您從存放區取得來源的 requirements.txt 檔案：

我們會在 requirements.txt 中加入 MCP Toolbox for Databases 的依附元件

<https://github.com/AbiramiSukumaran/renovation-agent-adk-mcp-toolbox/blob/main/requirements.txt>

2. 觀察您的 agent.py 檔案，其中包含存放區的程式碼：

我們將納入呼叫工具箱端點的工具，以便擷取特定訂購素材的訂單資料。

<https://github.com/AbiramiSukumaran/renovation-agent-adk-mcp-toolbox/blob/main/agent.py>

9. 模型設定

代理理解使用者要求和生成回覆的能力，是由大型語言模型 (LLM) 驅動。您的代理程式需要安全地呼叫這個外部 LLM 服務，因此需要驗證憑證。如果沒有有效的驗證，LLM 服務會拒絕代理程式的要求，代理程式也無法運作。

1. 從 [Google AI Studio](#) 取得 API 金鑰。
 2. 在下一個步驟中設定 `.env` 檔案時，請將 `<<your API KEY>>` 換成實際的 API 金鑰值。
-

10. 設定環境變數

1. 在範本 .env 檔案中設定參數值。以我的情況來說，.env 包含下列變數：

```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=<<your API KEY>>
GOOGLE_CLOUD_LOCATION=us-central1 <<or your region>>
GOOGLE_CLOUD_PROJECT=<<your project id>>
PROJECT_ID=<<your project id>>
GOOGLE_CLOUD_REGION=us-central1 <<or your region>>
```

將預留位置替換為您的值。

11. 執行代理程式

2. 使用終端機前往代理專案的上層目錄：

```
cd renovation-agent
```

3. 安裝依附元件：

```
pip install -r requirements.txt
```

4. 您可以在 Cloud Shell 終端機中執行下列指令，執行代理程式：

```
adk run .
```

5. 如要在 ADK 佈建的網頁版 UI 中執行，請執行下列指令：

```
adk web
```

6. 使用下列提示詞進行測試：

```
user>>
```

```
Hello. Check order status for Cement Bags.
```

12. 結果

```
user@abi-java-projects:~/adk-renovation-agent$ adk run .  
Log setup complete: /tmp/agents_log/agent.20250513_162955.log  
To access latest log: tail -F /tmp/agents_log/agent.latest.log  
Running agent adk_renovation_agent, type exit to exit.  
[user]: Hello, check status for cement bags  
[adk_renovation_agent]: The cement bags are Delivered.  
[user]: █
```

13. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

步驟 14 · 約 0 分鐘

14. 恭喜

恭喜！您已成功使用 ADK 和 MCP Toolbox for Databases 建立多代理應用程式！詳情請參閱產品說明文件：[Agent Development Kit](#) 和 [MCP Toolbox for Databases](#)。
